Programa:	**Atomica Deluxe 2.52**	
Protección:	Límite de 10 ejecuciones	
Objetivos:	Evitar que expire	
URL descarga:	www.popcap.com	
Dificultad:	Fácil	
Herramientas:	SoftIce + w32dasm y/o Ida Pro + ultraEdit32	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 02-10-04

Hola a todos otra vez..!!

Hoy volvemos a la carga con un juego cuya protección es la limitación de partidas que se pueden jugar que creo que son ***10***, nos da la posibilidad de introducir un código de registro el cual no sabemos claro está, y por el que nos pedirían unos 30€ siendo generosos, es decir que nos quieren clavar por el jueguecito. Si intentamos introducir un serial cualquiera y un nombre tres cuartos de lo mismo nos envía a paseo con una pantalla repleta de mensajes del tipo ***'Thet registration number does no appear, bla, bla…'***, o sea, que se da el negro de la situación en la que lo estamos poniendo. Además tiene un contador en el registro de windowz de las partidas jugadas hasta el momento, exactamente en:

HKEY_LOCAL_MACHINE\SOFTWARE\PopCap\Atomica

Pero tendremos una sorpresita si lo manoseamos, pues el programa dejará de funcionar de forma automática, tiene un ***CHECKSUM*** en el registro!

Como se puede apreciar la protección es cuando menos respetable, el programador se ha entretenido en ocultar las frases a miradas ajenas, y no digamos nada del checksum en el registro, ¿por dónde atacar?

Si desensamblamos el programa y echamos un vistazo la cosa no mejora, está lleno de saltos que no se sabe dónde acabarán, además las frases importantes del programa están ocultas, o se a que no podemos verlas con ***w32dasm*** pero curiosamente sí con ***IDA Pro*** increíble pero cierto, examinando el código detenidamente observamos que existe un salto mágico en ***:00426F6F*** , vemos algo tal que así: **;)**

```
* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:00426743(C), :0042674F(C), :00426F5A(C)
:00426F68 80BEC400000001      cmp byte ptr [esi+000000C4], 01   ;¿Es nuestro flag = 1?
:00426F6F 0F8577050000         jne 004274EC:004                  ; salto MALO!
```

luego, necesitamos que nuestro ***flag [esi+000000C4]*** esté a ***1*** en el momento de esta comparación, es por ello que deberá existir algún lugar en el código donde se inicialice a uno, lo buscamos, pulsando (en ***w32dasm***) en el icono de la linterna aceptamos y caemos en la siguiente zona: **;D**

```
* Referenced by a CALL at Address:
|:00427B6F      ;dirección desde donde se realizó la llamada!!
:00426230 56                  push esi
:00426231 8BF1                mov esi, ecx
:00426233 8B8EBC000000        mov ecx, dword ptr [esi+000000BC]
:00426239 57                  push edi
:0042623A C686C400000001      mov byte ptr [esi+000000C4], 01   /* asignación del flag a 1 */
```

como vemos está dentro de un call que es llamado desde :00427B6F, nos desplazamos a esa dirección con ***Shift+F12*** o en el menú ***Goto->Goto Code Locatio***' y caemos en:

```
* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00427A84(C)
:00427ABA 83F802              cmp eax, 00000002          ;comparación determinante.
:00427ABD 7553                jne 00427B12               ;salto BUENO!!
    . . .                     . . .
:00427AE1 C745FC00000000      mov [ebp-04], 00000000
:00427AE8 FF524C              call [edx+4C]
:00427AEB 837DF010            cmp dword ptr [ebp-10], 00000010     ;otra comparación interesante.
:00427AEF 0F8298000000        jb 00427B8D                          ;otro salto MALO que evita el call!!
:00427AF5 8B45DC              mov eax, dword ptr [ebp-24]
```

```
:00427AF8 50                    push eax
    •  •  •                      •  •  •
* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00427ABD(C)      ;dirección desde donde se ejecutó el salto!!
|
:00427B12 85C0                  test eax, eax
:00427B14 7577                  jne 00427B8D                ;salto MALO que evita el call!   ; (
:00427B16 8B4668                mov eax, dword ptr [esi+68]
    •  •  •                      •  •  •
:00427B29 FF5240                call [edx+40]
:00427B2C 84C0                  test al, al
:00427B2E 7455                  je 00427B85                 ;salto MALO que evita el call!   ; (
    •  •  •                      •  •  •
:00427B68 C680A003000001        mov byte ptr [eax+000003A0], 01
:00427B6F E8BCE6FFFF            call 00426230               ;nuestro call BUENO que inicializa el flag!!  ;-D
:00427B74 5E                    pop esi
```

Echando un vistazo al código anterior se observa que existen varios saltos que evitan la llamada ***BUENA*** en la que se inicializa el flag deseado, también hay saltos que te envían a la zona donde está éste, así pues si realizamos los siguientes cambios con ***ultraEdit32***:

```
:00427ABD 7553      jne 00427B12      POR:
:00427ABD EB53      jmp 00427B12

:00427B14 7577      jne 00427B8D      POR:
:00427B14 9090      nop (2 veces)

:00427B2E 7455      je 00427B85       POR:
:00427B2E 9090      nop (2 veces)
```

Aplicando los cambios con el editor hexadecimal y salvando el nuevo ejecutable con un nombre distinto al original conseguiremos nuestra propia versión libre de limitaciones, jeje..

Aunque existe un parche mejor.. entramos en el ***"call [edx+40]"*** y lo forzamos a retornar ***AL=0***, de esta manera nos evitamos tener que estar poniendo un serial cada vez que lo queramos utilizar ;-)

Y esto es todo por hoy, un crack sencillo donde los haya para que complicarse la visa verdad ;-)) S alguien quiere plantear sus dudas o bien hacer alguna crítica sabe que lo puede hacer en:

syxe00@yahoo.es
syxe00@hotmail.com

Como siempre digo todos los e-mails serán contestados… hasta otra, chao..!!